R GIS Data Science

Nepal Electoral Constituency GeoJSON

Scraping SVG, converting to GIS, and Georeferencing data with R

Scraping a constituency map from an election results site, converting its SVG paths to GIS geometry, and georeferencing it against ward boundaries in R.

Pukar Bhandari 2026-01-18

Table of contents

1 Introduction **1**
2 Setup and Libraries **2**
3 Scraping the SVG **2**
4 Parsing SVG Paths to Geometry **4**
5 Georeferencing (Affine Transformation) **6**
6 Reconstruction via Wards **8**

1 Introduction

In this workflow, we tackle a common geospatial problem: extracting data from stylized news graphics (like election heatmaps) and converting them into GIS-ready formats.

Rather than just cleaning the rough edges of a scraped SVG, we will use a **reconstruction technique**. We will:

1. **Scrape** the electoral map to get rough constituency shapes (Section 3, Section 4).
2. **Georeference** the shapes to align them with real coordinates (Section 5).
3. **Reconstruct** the precise boundaries by grouping official administrative "Wards" based on which constituency shape they fall into (Section 6).

The constituency map comes from the 2079 general election heatmap on ekantipur. Ward boundaries are the official administrative layer saved in this post's `data/` folder.

2 Setup and Libraries

We use `sf` for spatial operations and `rvest` for scraping. We also define paths for our inputs (official ward data) and outputs.

```
library(sf)       # Spatial data handling
library(xml2)     # Parsing XML/SVG
library(rvest)    # Web scraping
library(httr2)    # HTTP requests
library(stringr)  # String manipulation
library(dplyr)    # Data manipulation
library(here)     # Robust project-relative paths

post_dir ← here("posts", "nepal-election-geojson")

wards_geojson_path    ← here::here(post_dir,
"data", "nepal-wards.gpkg")
national_geojson_path ← here::here(post_dir,
"data", "nepal-national.gpkg")

output_svg_path       ← here::here(post_dir,
"output", "nepal_electoral_map.svg")
rough_map_path ← here::here(post_dir, "output",
"nepal_svg_rough.geojson")
output_geojson_path   ← here::here(post_dir,
"output", "nepal_electoral_map.geojson")

if (!dir.exists(dirname(output_svg_path)))
dir.create(dirname(output_svg_path), recursive =
TRUE)
```

Line 9
The path to this post's own folder, so the code runs from anywhere in the project.

Line 11
Official ward and national boundaries, used as the reference.

Line 14
Where the three outputs are written.

Line 18
Create the output folder if it is missing.

3 Scraping the SVG

We use `httr2` to fetch the webpage and `rvest` to extract the specific SVG element containing the

map. We also inject some CSS to ensure the map renders correctly if viewed in a browser (e.g., setting the viewBox and handling overflow).

> **The source site is not under your control**
>
> The code sends a browser-like user agent, because the site can refuse bare requests, and it falls back to the SVG saved by an earlier run if the site is down.

```
url ← "https://generalelection2079.ekantipur.com/
heatmap"

message("Fetching data from: ", url)

tryCatch({
  req ← request(url) ▷
    req_user_agent("Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36")

  resp ← req_perform(req)
  html_content ← resp_body_html(resp)

  svg_node ← html_element(html_content, xpath = "//
svg[.//pattern[@id='np__BG']]")

  if (!inherits(svg_node, "xml_missing")) {
    xml_set_attr(svg_node, "viewBox", "0 0 1920
1080")

    css_styles ← "
      path { fill: #cccccc; stroke: #fff; cursor:
pointer; stroke-width: 1.75px; }
      path:hover { opacity: 0.9; }
      .national-park { fill: #55e5a5 !important; }
      svg { overflow: visible !important; }
    "
    xml_add_child(svg_node, "style",
css_styles, .where = 0)

    write_xml(svg_node, output_svg_path)
    message("Success! Raw SVG saved to: ",
output_svg_path)

  } else {
    warning("SVG element not found on the page.")
  }
}, error = function(e) {
  message("Error fetching URL: ", e$message)
```

```
})

if (!exists("svg_node") &&
file.exists(output_svg_path)) {
  message("Using the previously saved SVG: ",
output_svg_path)
  svg_node ← read_xml(output_svg_path)
}
```

Line 7
Send a browser-like user agent, because the site can refuse bare requests.

Line 12
Pick the one `<svg>` that carries this map's background pattern.

Line 15
Fix the viewBox at 1920 by 1080 so every run uses the same coordinate system.

Line 23
Inject CSS so the saved SVG also looks right when opened in a browser.

Line 35
The source site is sometimes down: fall back to the SVG saved by an earlier run.

(The scraped SVG map is shown on the web version of this post.)

4 Parsing SVG Paths to Geometry

SVG paths are stored as strings in the `d` attribute. We need a robust parser to convert these strings into numerical coordinates. This function handles messy spacing and implicit commands often found in web SVGs.

Listing 1: Turning an SVG path string into polygon coordinates

```
parse_d_robust ← function(d) {
  d_clean ← str_replace_all(d, "-", " -")
  d_clean ← str_replace_all(d_clean, "[a-zA-Z]", "
")
  nums ← as.numeric(str_split(d_clean, "[\\s,]+")
[[1]])
  nums ← nums[!is.na(nums)]

  if(length(nums) < 4) return(NULL)

  mtx ← matrix(nums, ncol = 2, byrow = TRUE)
  if (!all(mtx[1,] == mtx[nrow(mtx),])) mtx ←
rbind(mtx, mtx[1,])
  if (nrow(mtx) < 4) return(NULL)

  return(mtx)
}

path_nodes ← xml_find_all(svg_node, "//*[local-
name()='path']")

polys_list ← list()
id_list ← character()

for (i in seq_along(path_nodes)) {
  d_attr ← xml_attr(path_nodes[i], "d")
  id_val ← xml_attr(path_nodes[i], "data-slug")

  if (is.na(id_val) || id_val == "") next

  coords ← parse_d_robust(d_attr)
  if (!is.null(coords)) {
    polys_list[[length(polys_list) + 1]] ←
st_polygon(list(coords))
    id_list ← c(id_list, id_val)
  }
}

if (length(polys_list) == 0) stop("No valid map
districts found after filtering.")

nepal_sf ← st_sf(constituency_id = id_list,
geometry = st_sfc(polys_list))
```

Line 2

Minified paths write negative numbers without a separator (`100-100`), so put a space before each minus sign.

Line 3
Remove the path commands (`M`, `L`, `C` and so on) and keep only the numbers.

Line 9
Pair the numbers up as x and y coordinates.

Line 10
Close the ring if the path is open; a polygon must end where it starts.

Line 25
Skip paths with no `data-slug`: they are buttons and background rectangles, not constituencies.

Line 36
The result is still in raw SVG coordinates, which is why the next step georeferences it.

The parser in Listing 1 is deliberately forgiving, because the paths come straight from a web page.

5 Georeferencing (Affine Transformation)

The extracted SVG uses screen coordinates (pixels). To make this map useful, we must align it with the real world (Longitude/Latitude).

We do this by calculating the bounding box of a reference map (`data/nepal-national.geojson`) and scaling our SVG to fit those dimensions.

```
if (!file.exists(national_geojson_path))
stop("Reference map not found.")

nepal_ref ← st_read(national_geojson_path, quiet =
TRUE)

target_bbox ← st_bbox(st_transform(nepal_ref,
3857))

flip_matrix ← matrix(c(1, 0, 0, -1), ncol = 2)
nepal_sf$geometry ← nepal_sf$geometry * flip_matrix

# Calculate Scale Factors
src_bbox ← st_bbox(nepal_sf)
src_w ← src_bbox$xmax - src_bbox$xmin
src_h ← src_bbox$ymax - src_bbox$ymin
```

```
tgt_w ← target_bbox$xmax - target_bbox$xmin
tgt_h ← target_bbox$ymax - target_bbox$ymin

nepal_sf$geometry ← nepal_sf$geometry -
c(src_bbox$xmin, src_bbox$ymin)

scale_matrix ← matrix(c(tgt_w / src_w, 0, 0,
tgt_h / src_h), ncol = 2)
nepal_sf$geometry ← nepal_sf$geometry *
scale_matrix

nepal_sf$geometry ← nepal_sf$geometry +
c(target_bbox$xmin, target_bbox$ymin)

st_crs(nepal_sf) ← 3857
```

Line 5
The reference map, projected to Web Mercator (EPSG:3857), gives the target size and aspect ratio for a web map.

Line 8
Flip the y-axis: SVG puts the origin at the top left, maps put it at the bottom left.

Line 18
Shift the SVG so its lower-left corner sits at (0, 0).

Line 21
Stretch it to the same width and height as the reference map.

Line 23
Shift it onto the reference map's position.

Line 25
Declare the coordinate system, now that the numbers really are Web Mercator metres.

```
# Create interactive map
mapview::mapview(nepal_sf, map.types =
"Esri.WorldGrayCanvas")
```

```
# Save the georeferenced SVG map before the ward
dissolve
# We use 'delete_dsn = TRUE' to overwrite any old
versions
st_write(
    nepal_sf,
    rough_map_path,
```

```
    driver = "GeoJSON",
    delete_dsn = TRUE,
    quiet = TRUE
)

message("Rough SVG map saved to: ", rough_map_path)
```

6 Reconstruction via Wards

The transformed SVG is roughly accurate but has "jagged" borders and low-quality topology. To fix this, we use the **Wards** method:

1. Convert high-quality Ward polygons to centroids.
2. Check which rough SVG shape each Ward centroid falls into.
3. Dissolve (merge) the Ward polygons based on that grouping.

> **The result**
>
> This results in a map that perfectly matches official administrative boundaries.

```
message("Reconstructing boundaries using Wards ... ")

if (!file.exists(wards_geojson_path)) stop("Wards
GeoJSON not found.")
wards <- st_read(wards_geojson_path, quiet = TRUE) |
>
  st_make_valid()

wards_utm <- st_transform(wards, 32645)
nepal_utm <- st_transform(nepal_sf, 32645) ▷
st_make_valid()

message("Matching wards to constituencies ... ")
sf_use_s2(FALSE)
ward_centroids <- st_centroid(wards_utm)
nearest_indices <-
st_nearest_feature(ward_centroids, nepal_utm)
wards_utm$constituency_id <-
nepal_utm$constituency_id[nearest_indices]

message("Dissolving geometries and removing
slivers ... ")
```

```
final_map_utm ← wards_utm ▷
  group_by(constituency_id) ▷
  summarise(geometry = st_union(geom)) ▷
  st_make_valid() ▷
  st_buffer(dist = 1) ▷
  st_buffer(dist = -1) ▷
  st_make_valid()

final_map ← st_transform(final_map_utm, 4326)
```

Line 7
Work in metres: Nepal falls mostly in UTM zone 45N (EPSG:32645), which keeps the cleanup accurate.

Line 11
Use the planar GEOS engine rather than spherical S2 for these joins.

Line 13
Each ward centroid is matched to the nearest rough constituency shape.

Line 20
Merge the wards that share a constituency into one polygon.

Line 22
Remove slivers by growing the shapes 1 metre and shrinking them back.

Line 26
Back to WGS84 (longitude and latitude) for the GeoJSON export.

```
# Create interactive map. The widget embeds every
vertex in the page, so map a
# simplified copy (200 m tolerance). The exported
GeoJSON below stays full resolution.
final_map_preview ← final_map_utm ▷
  sf::st_simplify(preserveTopology = TRUE,
dTolerance = 200) ▷
  st_transform(4326)

mapview::mapview(final_map_preview, map.types =
"Esri.WorldGrayCanvas")
```

```
# 6. Save Result
st_write(
    final_map,
    output_geojson_path,
```

```
    driver = "GeoJSON",
    layer_options = "COORDINATE_PRECISION=6",
    delete_dsn = TRUE,
    quiet = TRUE
)

message("Reconstruction Complete! Saved to: ",
output_geojson_path)
```

Download the output files

- nepal_electoral_map.svg
- nepal_svg_rough.geojson
- nepal_electoral_map.geojson